

ConvexHull

Rakesh Itani
Master's in Computer Science
Idaho State University
Pocatello, ID, 83029, USA
rakeshitani@isu.edu

February 2026

Abstract

This report summarizes the use of the divide and conquer algorithm to compute the convex hull of a set of points. The point set is recursively divided into smaller subproblems, convex hulls are computed for each half, and the results are merged by finding upper and lower tangents to produce the final hull. Pseudocode is provided and the asymptotic running time is analyzed. Experimental runtime measurements for increasing input sizes are also used to support the theoretical complexity.

1 Introduction

The convex hull of a set of point is defined as the smallest convex polygon, that encloses all of the points in the set. Convex means that the polygon has no corner that is bet inwards. The divide and conquer algorithm is a strategy of solving a large problem by breaking the problem into smaller sub problems, solving the sub-problems and finally combining them to get the desired output. To implement this algorithm, recursion will be used. Here, a divide and conquer algorithm will be used to compute the convex hull of a set of points. First, the set of points were divided into equal halves recursively and then those halves were merged together using the logics that solves the final convex hull of the given set of points.

2 Pseudocode and Asymptotic Analysis

2.1 Pseudocode to compute convex hull

Algorithm 1 Convex Hull

```
1: function CONVEXHULL(points)
2:   if len(points) ≤ 3 then
3:     return points
4:   mid = len(points) // 2
5:   left_hull ← CONVEXHULL(points[ : mid])
6:   right_hull ← CONVEXHULL(points[mid : ])
7:   right_most_left_hull ← maximum x-coordinate in left hull
8:   left_most_right_hull ← minimum x-coordinate in right hull
9:   left_decreasing = right_decreasing = False
10:  left_point, right_point = right_most_left_hull, left_most_right_hull
11:  slope ← SLOPE(right_most_left_hull, left_most_right_hull)
12:  while left_decreasing = False or right_decreasing = False do
13:    left_decreasing = right_decreasing = True
14:    while left_decreasing = True do
15:      left_point ← COUNTER_CLOCKWISE(left_hull, left_point)
16:      new_slope ← SLOPE(left_point, right_point)
17:      if new_slope < slope then
18:        slope = new_slope
19:      else
20:        left_point ← CLOCKWISE(left_hull, left_point)
21:        left_decreasing = False
22:      if SLOPE(left_point, CLOCKWISE(right_hull, right_point)) ≤
        slope then Break
```

```

23:     while right_decreasing = True do
24:         right_point  $\leftarrow$  CLOCKWISE(right_hull, right_point)
25:         new_slope  $\leftarrow$  SLOPE(left_point, right_point)
26:         if next_slope > slope then
27:             slope = new_slope
28:         else
29:             right_point  $\leftarrow$  COUNTER_CLOCKWISE(right_hull, right_point)
30:             right_decreasing = False
31:         if SLOPE(COUNTER_CLOCKWISE(left_hull, left_point), right_point)  $\geq$ 
slope then Break
32:     top_left_tan_point, top_left_tan_point = left_point, right_point
33:     left_decreasing = right_decreasing = False
34:     left_point, right_point = right_most_left_hull, left_most_right_hull
35:     slope  $\leftarrow$  SLOPE(right_most_left_hull, left_most_right_hull)
36:     while left_decreasing = False or right_decreasing = False do
37:         left_decreasing = right_decreasing = True
38:         while left_decreasing = True do
39:             left_point  $\leftarrow$  CLOCKWISE(left_hull, left_point)
40:             new_slope  $\leftarrow$  SLOPE(left_point, right_point)
41:             if next_slope > slope then
42:                 slope = new_slope
43:             else
44:                 left_point  $\leftarrow$  COUNTER_CLOCKWISE(left_hull, left_point)
45:                 left_decreasing = False
46:             if SLOPE(left_point, COUNTER_CLOCKWISE(right_hull, right_point))  $\geq$ 
slope then Break
47:         while right_decreasing = True do
48:             right_point  $\leftarrow$  COUNTER_CLOCKWISE(right_hull, right_point)
49:             new_slope  $\leftarrow$  SLOPE(left_point, right_point)
50:             if next_slope < slope then
51:                 slope = new_slope
52:             else
53:                 right_point  $\leftarrow$  CLOCKWISE(right_hull, right_point)
54:                 right_decreasing = False
55:             if SLOPE(CLOCKWISE(left_hull, left_point), right_point)  $\leq$ 
slope then Break

```

```

56:  lower_left_tan_point = left_point
57:  lower_right_tan_point = right_point
58:  final_hull = []
59:  right_hull_point = top_right_tan_point
60:  while right_hull_point  $\neq$  lower_right_tan_point do
61:      final_hull.APPEND(right_hull_point)
62:      right_hull_point  $\leftarrow$  CLOCKWISE(right_hull, right_point)
63:  final_hull.APPEND(lower_right_tan_point)
64:  left_hull_point = lower_left_tan_point
65:  while left_hull_point  $\neq$  top_left_tan_point do
66:      final_hull.APPEND(left_hull_point)
67:      left_hull_point  $\leftarrow$  CLOCKWISE(left_hull, left_point)
68:  final_hull.APPEND(lower_left_tan_point)
69:  return final_hull

```

Time complexity and space complexity

Given,

points: a list of tuples(x,y)

Space Complexity:

1. Since the depth of recursion for this algorithm is $O(\log n)$ space.
2. To store variables like left_hull, right_hull, final_hull is atmost the size of input. So, it takes $O(n)$ space.

$$S(n) = O(\log n) + O(n) \approx O(n) \quad (1)$$

Time Complexity:

1. The recursive division of a set of points into halves takes $T(n) = 2T(n/2) + O(n)$. Using Master Theorem, we get $O(n \log n)$
2. To find the left hull and right hull, i.e. dividing the points at each recursion, takes $O(n)$
3. To find the upper tangent, the while loop runs $O(n)$ times
4. To find the lower tangent, the while loop runs $O(n)$ times

5. Finally, to merge the hull points it takes $O(n)$ time

$$\begin{aligned} T(n) &= O(n \log n) + O(n) + O(n) + O(n) + O(n) \\ &\approx O(n * \log n) \end{aligned} \quad (2)$$

Analysis of algorithm:

The pseudocode takes set of points and outputs a single set. It uses divide and conquer algorithm to divide the given set of points into two halves recursively. Finally, it merges the hull points to get the final convex hull.

2.2 Pseudocode to get next point based on clockwise order

Algorithm 2 Get points on Clockwise order

```

1: function CLOCKWISE(points, point)
2:   index  $\leftarrow$  index of the point in points
3:   if index = len(points) - 1 then
4:     return points[0]
5:   else
6:     return points[index + 1]

```

Time complexity and space complexity

Given,

points: a set of points

point: a valid tuple(x,y)

Space Complexity:

$$S(n) = O(1) + O(1) + O(1) = O(3) \approx O(1) \quad (3)$$

Time Complexity:

$$\begin{aligned} S(n) &= O(1) + O(1) + O(1) + O(1) + O(1) + O(1) \\ &= O(6) \approx O(1) \end{aligned} \quad (4)$$

Analysis of algorithm:

The pseudocode takes two inputs: a set of points and a valid tuple(x,y) and outputs a tuple(x,y). It returns next point in clockwise order of given point in the set of points.

2.3 Pseudocode to get next point based on counterclockwise order

Algorithm 3 Get points on Counter-Clockwise order

```
1: function COUNTER_CLOCKWISE(points, point)
2:   index  $\leftarrow$  index of the point in points
3:   if index = 0 then
4:     return points[len(points) - 1]
5:   else
6:     return points[index - 1]
```

Time complexity and space complexity

Given,

points: a set of points

point: a valid tuple(x,y)

Space Complexity:

$$S(n) = O(1) + O(1) + O(1) = O(3) \approx O(1) \quad (5)$$

Time Complexity:

$$\begin{aligned} S(n) &= O(1) + O(1) + O(1) + O(1) + O(1) + O(1) \\ &= O(6) \approx O(1) \end{aligned} \quad (6)$$

Analysis of algorithm:

The pseudocode takes two inputs: a set of points and a valid tuple(x,y) and outputs a tuple(x,y). It returns next point in counter-clockwise order of given point in the set of points.

2.4 Pseudocode to compute slope

Algorithm 4 Slope

```
1: function SLOPE(point1, point2)
2:    $x_1, y_1, x_2, y_2 = \text{point1}[0], \text{point1}[1], \text{point2}[0], \text{point2}[1]$ 
3:   return  $((\text{point2}[1] - \text{point1}[1]) / (\text{point2}[0] - \text{point1}[0]))$ 
```

Time complexity and space complexity

Given,

point1: a valid tuple(x,y)

point2: a valid tuple(x,y)

Space Complexity:

$$S(n) = O(1) + O(1) + O(1) + O(1) + O(1) = O(5) \approx O(1) \quad (7)$$

Time Complexity:

$$T(n) = O(1) + O(1) \approx O(1) \quad (8)$$

Analysis of algorithm:

The pseudocode takes two points as input and outputs a single value. It computes the slope between two points and return the result.

3 Observations and Results

Sample Size	Distribution	Average Time in Seconds
10	Spherical	0.0000 sec
100	Spherical	0.0014 sec
1000	Spherical	0.0086 sec
10000	Spherical	0.0472 sec
100000	Spherical	0.3674 sec
500000	Spherical	1.7106 sec
1000000	Spherical	3.4452 sec

Here, for the experimentation to observe time elapsed, spherical distribution was chosen to collect the elapsed time which was the time taken to solve the convex hull for 7 different sample sizes as shown in the table above. It is clear from the table that the elapsed time increases as the sample size increases.

To give more insight on this, 5 different samples were observed for each sample size to make sure the observations were consistent. The average time in seconds as shown in the table above is the average time taken by each sample throughout 5 repetitions. The data obtained was used to plot the following graph.

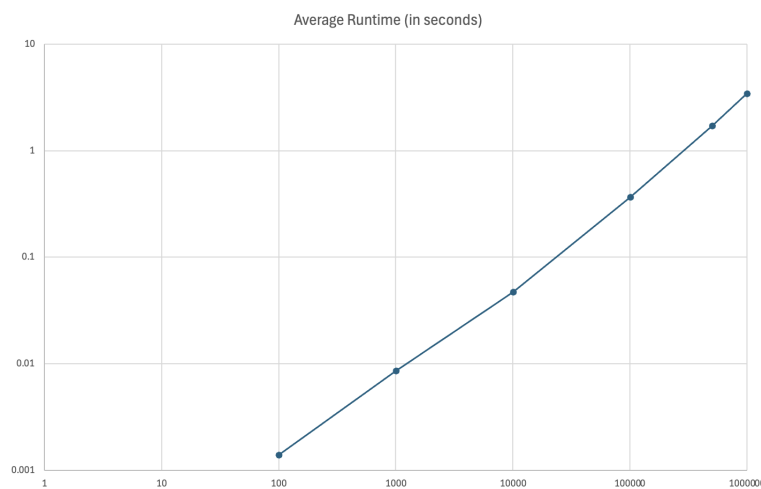


Figure 1: n vs Average Runtime for 5 repetitions

In the graph (Figure 1), we can observe an increasing trend showing that the mean time required to compute the convex hull increases as the number of input points increases. The graph was plotted using logarithmic scale on both the x-axis and y-axis. This scaling helps visualize the growth more clearly, since the input sizes increase exponentially (10, 100, 1000, etc.), and logarithmic scaling makes the trend easier to interpret.

When plotted on a normal linear scale, the curve appeared steep and difficult to analyze. However, after applying logarithmic scaling, the graph shows a smooth and predictable increase consistent with the theoretical time complexity.

Based on the theoretical analysis, the time complexity of the CONVEXHULL algorithm is $O(n \log n)$. The empirical results support this theoretical behavior. Although the actual measured runtime values are smaller than the theoretical upper bound, this difference is expected because asymptotic analysis describes growth rate rather than exact runtime. The constant factor of proportionality depends on implementation details, hardware performance, and system overhead.

Overall, both the theoretical analysis and experimental results confirm that the divide and conquer convex hull algorithm runs in $O(n \log n)$ time.

3.1 Results

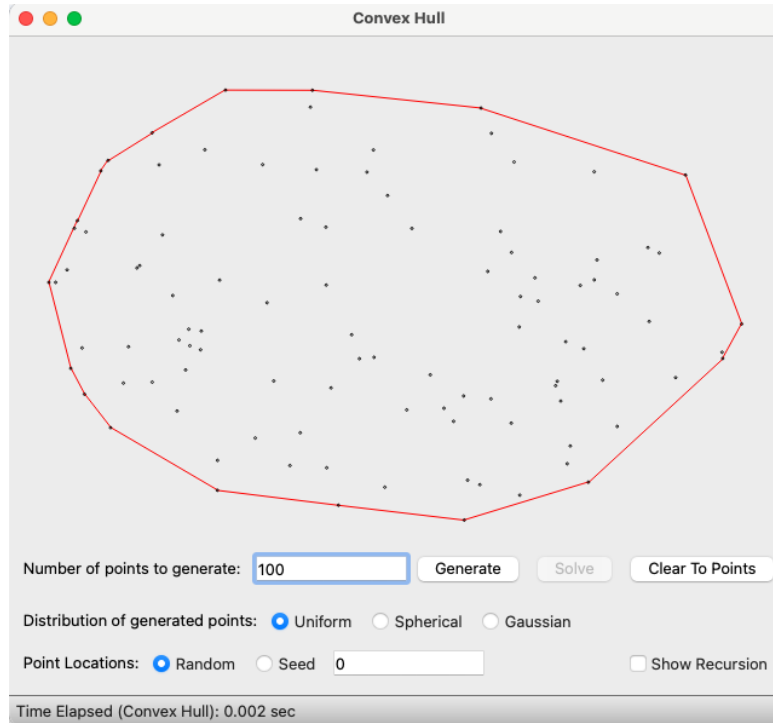


Figure 2: Convex Hull for 100 points

Figure 2 shows the solving of the convex hull plot for 100 sample size. As expected, the program was able to compute and display the convex hull for uniform distribution. The program took 0.002 seconds to compute the convex hull under the uniform distribution and random seed criteria as show in the figure.

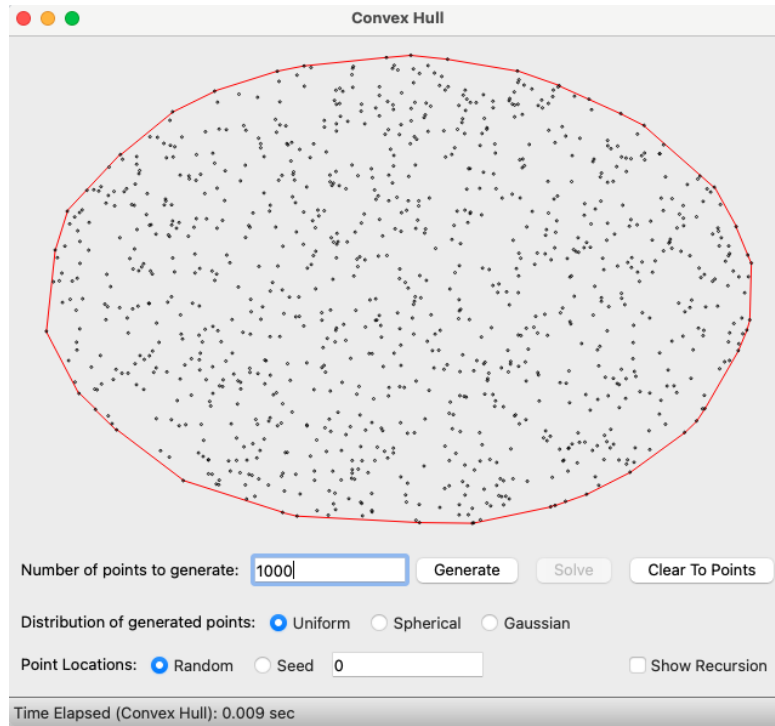


Figure 3: Convex Hull for 1000 points

Again, 1000 random points were used to test and compute the convex hull for uniform distribution under random seed. It took roughly 0.009 seconds, which lines up with our observation from earlier that the elapsed time increases with increase in sample size.

4 Conclusion

In Summary, the convex hull was computed using divide and conquer algorithm along with the orientation and distance. The analysis of pseudocode allowed for the computation of time and space complexity.